Student B number: **B300238**

Hi!
Some feedback for you for the OptionalICA, part of which is shown here:

Tasks overview
Write a generic python programme/script

- **that doesn't use BioPython**
- **that isn't just a (series of) Unix/bash script(s) called from python...**
- **that you haven't used ChatGPT or other AI tools for...**
- that you will make available as a passworded `ccrypt` encrypted file in a PUBLIC repository on GitHub in your Bxxxxxx-2025 GitHub account

that will allow the user:

1. to identify a family of protein sequences from a user-defined subset of the taxonomic tree (e.g. glucose-6-phosphatase proteins from *Aves* (birds), or ABC transporters in mammals, or kinases in rodents, or adenyl cyclases in vertebrates *etc.*) that could then be processed using, for example, one or more of the EMBOSS programmes installed on the MSc server:
   - to determine, and plot, the level of protein sequence conservation across the species within that taxonomic group
   - to scan the protein sequence(s) of interest with motifs from the PROSITE database, to determine whether any known motifs (domains) are associated with this subset of sequences
   - to do any other appropriate EMBOSS (or other) analysis that you think might add relevant biological information to the outputs
2. write a "help manual" for your programme which has **two** sections:
   - one aimed at an "ordinary user" (non-coder)
   - one aimed at a competent python code-writer

A possible answer

Like the first Optional ICA, the emphasis of this ICA was for you to write something that is actually useful, complete with git repository, that could be used by you or others in the future.

It should use some, if not all (and more!), of the things we did in the Python part of the course, so that includes variables, lists, loops, dictionaries, functions, regex, dataframes perhaps, and so on... apart from Biopython!

Code comments

- **Could I clone the GitHub repository asked for?**
  Yes, no problems!
- **Date/time on GitHub repo**

  git log | grep Date

  ```
  Date:   Sun Nov 23 23:47:09 2025 +0000
  Date:   Sun Nov 23 18:33:48 2025 +0000
  Date:   Sun Nov 23 13:19:34 2025 +0000
  Date:   Sun Nov 23 12:50:15 2025 +0000
  Date:   Sun Nov 23 11:22:24 2025 +0000
  ```
- **Did the GitHub repository only include the pipeline code as a tar.gz filefile, as requested?**
  Yes

  ls -1

  ```
  B300238-2025.OptionalPythonICA.tar.gz.cpt
  ```
- **Could I decrypt the file with the password given?**
  Yes, worked fine!
- **Was your user name visible anywhere?**

  grep Author *things* | cut -d ' ' -f2,3 | sort | uniq

  ```
  B300238-2025 <email_not_provided>
  ```

  git log | grep Au

  ```
  Author: B300238-2025 <email_not_provided>
  Author: B300238-2025 <email_not_provided>
  Author: B300238-2025 <email_not_provided>
  ```

```
Author: B300238-2025 <email_not_provided>
Author: B300238-2025 <email_not_provided>
```

No, couldn't find it at all: well done!

- **What did the git log suggest to me?**

```
egrep "Date" *things* | awk '{if(NF>1){print $2,$3,$4}}' | uniq -c
```

```
egrep "Date" *things* | awk '{if(NF>1){print $0}}' | uniq -c
```

```
egrep -v "Auth|Date|commit" *things* | awk '(NF>=1)'
```

Dates

```
  3 Sun Nov 23
  3 Sat Nov 22
  1 Thu Nov 20
  1 Wed Nov 19
  1 Tue Nov 18
  1 Mon Nov 17
  1 Sat Nov 15
```

Messages

```
23 Nov 2025: Tweaked makeblastdb to use records instead of raw file.
23 Nov 2025: Minor tweak to recommended range. Fixed prosite_locations.tsv not outputting properly.
23 Nov 2025: Corrected requirements.txt.
22 Nov 2025: Added comments.
22 Nov 2025: Done blast_analysis. Added plots. Installed matplotlib and seaborn.
22 Nov 2025: Done scan_prosite. Updated main, conservation_analysis. Added more user interactivity. Installed pandas.
20 Nov 2025: Done conservation_analysis. Edited main, blast_analysis, scan_prosite.
19 Nov 2025: Done parse_fasta. Updated all except input_handler.
18 Nov 2025: Done fetch_sequence. Updated main and input_handler.
17 Nov 2025: Done input_handler. Updated main. Added .gitignore and requirements (.venv).
15 Nov 25: Created repo
```

Coding was done over multiple sessions, good use of messages. Interesting inclusion of date in the message...

- **# System check**

```
python3 --version
```

Python 3.12.3

```
which python3
```

/usr/bin/python3

- **What modules were used?**

```
cat *.py | grep "import " | sort | uniq -c
```

```
      1 from blast_analysis import make_blast_db, select_blast_ref, run_blast_search, parse_
      1 from conservation_analysis import build_clustalo_input, run_clustalo, run_plotcon, g
      1 from fetch_sequence import run_esearch, run_efetch
      1 from input_handler import validate_protein, validate_taxon
      1 from parse_fasta import parse_fasta
      1 from scan_prosite import parse_clu_results, build_prosite_input, run_prosite_scan, p
      1 import glob
      1 import json
      3 import matplotlib.pyplot as plt
      3 import pandas as pd
      1 import re
      2 import requests
      3 import seaborn as sns
      5 import subprocess
      2 import sys
```

- **Code comments: I tried to run the programme on the `msc8 server` without consulting the user / maintenance manual**

BECAUSE THAT IS WHAT MOST USERS WILL DO...!

 # Command line versions of example runs done 25 Nov 2025

```
esearch -db protein -query 'Aves[organism] AND glucose-6-phosphatase[protein name] NOT partial'
```

```
<ENTREZ_DIRECT>
  <Db>protein</Db>
  <WebEnv>MCID_637e7ed1046db471fd3c3a3b</WebEnv>
  <QueryKey>1</QueryKey>
  <Count>52</Count>
  <Step>1</Step>
  </ENTREZ_DIRECT>
```

```
esearch -db protein -query 'Ascomycota[organism] AND pyruvate dehydrogenase[protein name] NOT partial'
```

```
<ENTREZ_DIRECT>
  <Db>protein</Db>
  <WebEnv>MCID_655bb7354a2fb649ec525ed3</WebEnv>
  <QueryKey>1</QueryKey>
  <Count>228</Count>
  <Step>1</Step>
  </ENTREZ_DIRECT>
```

```
# A large dataset (Eukaryota kinase) txid2759
esearch -db protein -query 'Eukaryota[organism] AND kinase[protein name] NOT partial'
<ENTREZ_DIRECT>
  <Db>protein</Db>
  <WebEnv>MCID_637e7ed1046db471fd3c3a3b</WebEnv>
  <QueryKey>1</QueryKey>
  <Count>3098</Count>
  <Step>1</Step>
  </ENTREZ_DIRECT>
```

- **Could I run the script directly as a script (did it have a shebang line?)?**
  Yes, but I had to work out which one to use! If you have multiple scripts, do make it clear which one should be run first, so that everyone could work it out, e.g. I guessed it might be `main.py` to start the process...
  Yes, it ran to the end.

```
# Run 1
python3 main.py

Traceback (most recent call last):
  File "/localdisk/home/aivens2/..../B300238-2025/main.py", line 8, in <module>
    import pandas as pd
ModuleNotFoundError: No module named 'pandas'

REASON: pandas error... do we need to have an environment?

Would it not be a good to have a try except thing here....?
```

```
# What files do we have?
ls -R
all_the_things_I_did_for_OptionalPythonICA
B300238-2025.OptionalPythonICA.tar.gz
blast_analysis.py
conservation_analysis.py
fetch_sequence.py
input_handler.py
main.py
parse_fasta.py
requirements.txt
scan_prosite.py
```

```
# Run 2: I HAD to RTFM...
# Inside the terminal, type the following command lines
python3 -m venv .venv
source .venv/bin/activate
```

```
pip install -r requirements.txt
```

```
  ....
Successfully installed certifi-2025.11.12 charset-normalizer-3.4.4 contourpy-1.3.3 cycler-0.
```

```
python3 main.py
```

```
python3 main.py
Your email (needed for NCBI URL request): al.ivens@ed.ac.uk
Protein family: glucose-6-phosphatase
Taxon group: Aves

Success! Your protein family: PAP2      Pfam ID: PF01569
Success! Your taxon group: Aves Taxon ID: 8782

Searching NCBI Protein...
Your dataset contains more than 1000 protein sequences!
Are you sure you want to continue? (y/n) n

Pipeline stopped!
```

```
REASON: I asked to leave.  WHAT is it finding?  There's only 52 of them!
```

I wasnt quite sure how your query was being formed, which fields of the NCBI database was it looking in?

```
# Query looking in the protein name field, but excluding partials
esearch -db protein -query 'Aves[organism] AND glucose-6-phosphatase[protein name] NOT partial' | grep Count
<Count>52</Count>
# Query not looking in the protein name field, but excluding partials
esearch -db protein -query 'Aves[organism] AND glucose-6-phosphatase NOT partial' | grep Count
<Count>1136</Count>
```

```
# Query not looking in the protein name field, include partials
esearch -db protein -query 'Aves[organism] AND glucose-6-phosphatase' | grep Count
<Count>1491</Count>
```

I'll try saying yes....

```
# Run 3
python3 main.py
Your email (needed for NCBI URL request): al.ivens@ed.ac.uk
Protein family: glucose-6-phosphatase
Taxon group: Aves

Success! Your protein family: PAP2      Pfam ID: PF01569
Success! Your taxon group: Aves Taxon ID: 8782

Searching NCBI Protein...
Your dataset contains more than 1000 protein sequences!
Are you sure you want to continue? (y/n) y
Protein UIDs successfully retrieved!

Fetching all protein sequences in FASTA format...
FASTA sequences successfully downloaded to PAP2_Aves_rawsequences.fasta

Parsing PAP2_Aves_rawsequences.fasta...
Successfully parsed PAP2_Aves_rawsequences.fasta

Please input a number for the number of sequences to be analysed with Clustal0. Maximum: 499
```

```
The top few sequences with the largest lengths will be used for conservation analysis.
Note: The larger the number, the slower the analysis. Recommended: 500-1000
Sample size: 500

Your sample size: 500
The 500 longest protein sequences will be used for conservation analysis.

Performing conservation analysis with ClustalO and Plotcon...
ClustalO output successfully generated! Generated PAP2_Aves_clustalo_out.clu
Outputting PAP2_Aves_clustalo_plotcon_graph to terminal. Check graph, and close to continue!
X connection to localhost:13.0 broken (explicit kill or server shutdown).
Plotcon output successfully generated! Generated PAP2_Aves_clustalo_plotcon_graph.1.png, PAP
Successfully obtained consensus sequence! Generated PAP2_Aves_clustalo_consensus.fasta
Successfully obtained infoalign results for PAP2_Aves_clustalo_out.clu! Generated PAP2_Aves_

Please input a number for the number of sequences to be scanned for PROSITE motifs. Maximum:
The top few sequences with the highest similarity with the consensus sequence will be scanne
Scan size: 500

Your scan size: 500
The 500 protein sequences with the highest similarity to the consensus sequence will be scan

Scanning protein sequences for PROSITE motifs...
PROSITE output successfully generated! Generated 450 .patmatmotifs files.
Generating count_plot_motifs.png...
Outputting Count Plot of PROSITE Motifs Found to terminal. Check plot, and close to continue
Generated prosite_scan_summary.tsv! PROSITE motifs found:
['AMIDATION', 'RGD', 'LEUCINE_ZIPPER', 'SUBTILASE_HIS', 'THIOL_PROTEASE_HIS']

Running BLAST on protein sequences...


Building a new DB, current time: 12/09/2025 20:21:57
New DB name:   /localdisk/home/aivens2/...../PAP2_Aves/PAP2_Aves_blast/PAP2_Aves_blast
New DB title:  PAP2_Aves/PAP2_Aves_blast/PAP2_Aves_records.fasta
Sequence type: Protein
Keep MBits: T
Maximum file size: 3000000000B
Adding sequences from FASTA; added 4999 sequences in 0.0823519 seconds.


Using XP_062486722.1_blast_ref.fasta as BLAST reference sequence...
Successfully completed BLAST analysis! Generated PAP2_Aves_blastoutput.out
Generating scatterplot_blast.png...
Outputting Scatterplot of BLAST Hits to terminal. Check plot, and close to continue!

Full analysis successfully completed! Check PAP2_Aves for all outputs.
```

```
REASON: Complete, good pics!
```

 # Run 4, a query with a space in it, there should be 228 of them

python3 main.py

```
Your email (needed for NCBI URL request): al.ivens@ed.ac.uk
Protein family: pyruvate dehydrogenase
Taxon group: Ascomycota

Success! Your protein family: PDH_E1_M  Pfam ID: PF17831
Success! Your taxon group: Ascomycota   Taxon ID: 4890

Searching NCBI Protein...
No result for (PDH_E1_M) AND txid4890[organism] NOT partial on NCBI Protein!

No UIDs could be retrieved from NCBI Protein! Please try again.
```

```
REASON: this code is looking in the wrong place for sequences to download?
```

 # Run 5: run with something that gives no proteins...
 # These are MY results from 2024

source

esearch -db protein -query 'Klingon[organism] AND green slime[protein name] NOT partial'

```
WARNING:  FAILURE ( Mon 25 Nov 20:15:14 GMT 2024 )
nquire -url https://eutils.ncbi.nlm.nih.gov/entrez/eutils/ esearch.fcgi -retmax 0 -usehistor
<ErrorList>
  <PhraseNotFound>Klingon[organism]</PhraseNotFound>
  <PhraseNotFound>green slime[protein name]</PhraseNotFound>
<ErrorList>
SECOND ATTEMPT
 WARNING:  FAILURE ( Mon 25 Nov 20:15:16 GMT 2025 )
nquire -url https://eutils.ncbi.nlm.nih.gov/entrez/eutils/ esearch.fcgi -retmax 0 -usehistor
<ErrorList>
  <PhraseNotFound>Klingon[organism]</PhraseNotFound>
  <PhraseNotFound>green slime[protein name]</PhraseNotFound>
<ErrorList>
LAST ATTEMPT
 ERROR:  FAILURE ( Mon 25 Nov 20:15:17 GMT 2025 )
nquire -url https://eutils.ncbi.nlm.nih.gov/entrez/eutils/ esearch.fcgi -retmax 0 -usehistor
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE eSearchResult>
<eSearchResult>
  <Count>0</Count>
  <RetMax>0</RetMax>
  <RetStart>0</RetStart>
  <QueryKey>1</QueryKey>
  <WebEnv>MCID_637e7f5635097c618c5a7faf</WebEnv>
  <QueryTranslation>Klingon[organism] AND green slime[protein name] NOT partial[All Fields]<
  <ErrorList>
    <PhraseNotFound>Klingon[organism]</PhraseNotFound>
    <PhraseNotFound>green slime[protein name]</PhraseNotFound>
  </ErrorList>
  <WarningList>
    <OutputMessage>No items found.</OutputMessage>
  </WarningList>
</eSearchResult>
QUERY FAILURE
<ENTREZ_DIRECT>
  <Db>protein</Db>
  <WebEnv>MCID_637e7f58d1b1ae6f00693445</WebEnv>
  <QueryKey>1</QueryKey>
  <Count>0</Count>
  <Step>1</Step>
</ENTREZ_DIRECT>
```

 # Klingon[organism] AND green slime[protein name] NOT partial

python3 main.py

```
Your email (needed for NCBI URL request): al.ivens@ed.ac.uk
Protein family: green slime

Please input a valid protein family. No result for green slime on EBI Search!
Protein family: meh

Please input a valid protein family. No result for meh on EBI Search!
Protein family:
```

```
REASON: didnt find any, good!
```

- **Did the script ask/tell me where I wanted to put files while it was running/when it was finished?**
  Yes, good!
- **Could I run the script directly in my home space (i.e. no links to places I can't get to easily without immediately knowing who you are)?**
  Yes, seemed to find everything, but ONLY after I set up an environment!
- **Was it actually a Python programme as requested, and not just a short Python wrapper for a Unix script?**
  Yes, definitely Python
- **Did the code have lots of comments so I could see what the code was doing?**
  Yes, more than sufficient, excellent in fact!
- **Was it logically laid out?**
  Yes
- **Was there evidence that a flow-chart or similar had been used in planning the design (i.e. output from one bit being the input for the next)?**
  Yes
- **Were there any errors, and if so, what caused them?**
  Did not break under normal use
- **Which components of python were used?**

```
../grepcheck | awk '{gsub("\t","",$0); gsub(" "," ",$0); gsub(" "," ",$0); print $0}' > gcheck.out
```

```
cat gcheck.out
```

 # Looking for import statements

```
27
  1 from blast_analysis import make_blast_db, select_blast_ref, run_blast_search, parse_blas
  1 from conservation_analysis import build_clustalo_input, run_clustalo, run_plotcon, get_c
  1 from fetch_sequence import run_esearch, run_efetch
  1 from input_handler import validate_protein, validate_taxon
  1 from parse_fasta import parse_fasta
  1 from scan_prosite import parse_clu_results, build_prosite_input, run_prosite_scan, parse
  1 import glob
  1 import json
  3 import matplotlib.pyplot as plt
  3 import pandas as pd
  1 import re
  2 import requests
  3 import seaborn as sns
  5 import subprocess
  2 import sys
```

 # Looking for SeqIO|biopython

```
0
```

 # Looking for os

```
0
```

 # Looking for os.system

```
0
```

 # Looking for os.remove

```
0
```

 # Looking for subprocess

```
14
  1 blast_analysis.py:    subprocess.call(f"blastp -db {out_dir}/{blast_db}/{blast_db} -query {out_dir}/{blast_db}/{blastref_f:
  1 blast_analysis.py:    subprocess.call(f"makeblastdb -in {out_dir}/{blast_db}/{blast_db_input} -dbtype prot -out {out_dir}/
  1 conservation_analysis.py:    subprocess.call(f"clustalo -i {out_dir}/{out_clustalo}/{clustalo_input_filename} -o {out_dir},
  1 conservation_analysis.py:    subprocess.call(f"cons -sequence {out_dir}/{out_clustalo}/{clustalo_output_filename} -outseq
  1 conservation_analysis.py:    subprocess.call(f"infoalign {out_dir}/{out_clustalo}/{clustalo_output_filename} -outfile {out_
  1 conservation_analysis.py:    subprocess.call(f"plotcon -sequence {out_dir}/{out_clustalo}/{clustalo_output_filename} -grapl
  1 conservation_analysis.py:    subprocess.call(f"plotcon -sequence {out_dir}/{out_clustalo}/{clustalo_output_filename} -grapl
```

```
  1 conservation_analysis.py:    subprocess.call(f"plotcon -sequence {out_dir}/{out_clustalo}/{clustalo_output_filename} -grapl
  1 main.py:    subprocess.call(f"mkdir -p {out_dir}/{blast_db}", shell = True)
  1 main.py:    subprocess.call(f"mkdir -p {out_dir}/{out_clustalo}", shell = True)
  1 main.py:    subprocess.call(f"mkdir -p {out_dir}/{out_prosite}", shell = True)
  1 main.py:    subprocess.call(f"mkdir -p {out_dir}", shell = True)
  1 main.py:    subprocess.call(f"rm -rf {out_dir}", shell = True)
  1 scan_prosite.py:    subprocess.call(f"patmatmotifs -sequence {out_dir}/{out_prosite}/{prosite_input} -outfile {out_dir}/{ou
```

Looking for write

```
11
```

Looking for dictionaries

```
3
fetch_sequence.py:    ids_list = search_result.get("esearchresult", {}).get("idlist", [])
input_handler.py:    id_list = search_result.get("esearchresult", {}).get("idlist", [])
input_handler.py:    taxon_name = summary.get("result", {}).get(taxon_id, {}).get("scientifi
```

Looking for matplot use

```
15
```

Looking for functions

```
20
blast_analysis.py:def make_blast_db(out_dir, records, blast_db):
blast_analysis.py:def parse_blast_output(out_dir, blast_db, blastresults_filename):
blast_analysis.py:def run_blast_search(out_dir, blastref_filename, blast_db):
blast_analysis.py:def select_blast_ref(top_records, top_df, out_dir, blast_db):
conservation_analysis.py:def build_clustalo_input(records, out_dir, out_clustalo, size):
conservation_analysis.py:def get_consensus(out_dir, out_clustalo, clustalo_output_filename):
conservation_analysis.py:def get_infoalign(out_dir, out_clustalo, clustalo_output_filename):
conservation_analysis.py:def run_clustalo(out_dir, out_clustalo, clustalo_input_filename):
conservation_analysis.py:def run_plotcon(out_dir, out_clustalo, clustalo_output_filename):
fetch_sequence.py:def run_efetch(ids_list, out_dir, email):
fetch_sequence.py:def run_esearch(protein_family, taxon_id, email):
input_handler.py:def validate_protein(protein_family):
input_handler.py:def validate_taxon(taxon_group, email):
main.py:def main():
parse_fasta.py:    def mapper(entry):
parse_fasta.py:def parse_fasta(out_dir, fasta_filename):
scan_prosite.py:def build_prosite_input(records_top, out_dir, out_prosite):
scan_prosite.py:def parse_clu_results(records, out_dir, out_clustalo, infoalign_results, out_prosite, size):
scan_prosite.py:def parse_prosite_output(out_dir, out_prosite, prosite_output_list):
scan_prosite.py:def run_prosite_scan(out_dir, out_prosite, prosite_input_list):
```

Looking for for loops

```
23
```

Looking for ifs

```
25
```

Looking for lists

```
7
```

Looking for arrays

```
0
```

Looking for regex

```
2
    email, re.IGNORECASE
    if re.search(
```

Looking for pandas

```
10
blast_analysis.py:    blast_df = pd.read_csv(f"{out_dir}/{blast_db}/{blastresults_filename}", sep = "\t", comment = "#", names
blast_analysis.py:# Function: Parse BLAST output file to a pandas dataframe, and visualise BLAST output by plotting a scatterp
blast_analysis.py:import pandas as pd
blast_analysis.py:# Output: Scatterplot of BLAST hits and parsed BLAST output to pandas dataframe.
main.py:import pandas as pd
main.py:    # parse_blast_output() takes in 3 args: out_dir, blast_db and BLAST output file. Output: Scatterplot of BLAST hits
scan_prosite.py:    df = pd.read_csv(f"{out_dir}/{out_prosite}/{file}", sep = "\t", comment = "#", na_values = [""])
scan_prosite.py:import pandas as pd
```

```
scan_prosite.py:    infoalign_df = pd.read_csv(f"{out_dir}/{out_clustalo}/{infoalign_results}", sep = "\t", na_values = [""], l
scan_prosite.py:    summary_df = pd.concat(df_list, join = "outer", ignore_index = True)
```

Looking for try/except

```
blast_analysis.py:0 conservation_analysis.py:0 fetch_sequence.py:0 input_handler.py:0 main.py:0 parse_fasta.py:0 scan_prosite.p
blast_analysis.py:    for entry in records:
blast_analysis.py:    file.write(f">{entry['uid']} {entry['description']} [{entry['species']}]\n")
blast_analysis.py:    file.write(f"{entry['sequence']}\n\n")
conservation_analysis.py:    for entry in records_sorted:
conservation_analysis.py:    file.write(f">{entry['uid']} {entry['description']} [{entry['species']}]\n")
conservation_analysis.py:    file.write(f"{entry['sequence']}\n\n")
fetch_sequence.py:    sys.exit("Please try again.")
main.py:    try:
main.py:    except Exception as e:
main.py:    try:
main.py:    except Exception as e:
main.py:    sys.exit("\nNo UIDs could be retrieved from NCBI Protein! Please try again.")
main.py:    try:
main.py:    except Exception as e:
main.py:    try:
main.py:    try:
main.py:    except Exception as e:
main.py:    print("\nRetrying...")
main.py:    except Exception as e:
main.py:    try:
main.py:    except Exception as e:
parse_fasta.py:    fasta_list = [entry.strip() for entry in fasta_text.split(">") if entry.strip()]
parse_fasta.py:    def mapper(entry):
parse_fasta.py:    entry = entry.replace("\n", "")
parse_fasta.py:    parts = entry.split("]", 1)
scan_prosite.py:    for entry in records_top:
scan_prosite.py:    with open(f"{out_dir}/{out_prosite}/{entry['uid']}.fasta", "w") as file:
scan_prosite.py:    file.write(f">{entry['uid']} {entry['description']} [{entry['species']}]\n")
scan_prosite.py:    file.write(f"{entry['sequence']}\n\n")
scan_prosite.py:    prosite_input.append(f"{entry['uid']}")
```

Checking use of esearch

```
30
```

Checking use of clustalo

```
13
conservation_analysis.py:# build_clustalo_input() takes in 4 args: Parsed sequences to JSON, out_dir, out_clustalo and user's i
conservation_analysis.py:# Function: Run clustalo command line.
conservation_analysis.py:# get_consensus() takes in 3 args: out_dir, out_clustalo and MSA output.
conservation_analysis.py:# get_infoalign() takes in 3 args: out_dir, out_clustalo and MSA output.
conservation_analysis.py:# run_clustalo() takes in 3 args: out_dir, out_clustalo and input FASTA filename for ClustalO.
conservation_analysis.py:# run_plotcon() takes in 3 args: out_dir, out_clustalo and MSA output.
conservation_analysis.py:    subprocess.call(f"clustalo -i {out_dir}/{out_clustalo}/{clustalo_input_filename} -o {out_dir}/{out
main.py:    # build_clustalo_input() takes in 4 args: Parsed sequences to JSON, out_dir, out_clustalo and user's input for samp
main.py:    # get_consensus() takes in 3 args: out_dir, out_clustalo and MSA output. Output: Consensus sequence FASTA filename
main.py:    # get_infoalign() takes in 3 args: out_dir, out_clustalo and MSA output. Output: infoalign results .txt filename.
main.py:    out_clustalo = f"{out_dir}_clustalo"
main.py:    # run_clustalo() takes in 3 args: out_dir, out_clustalo and input FASTA filename for ClustalO. Output: Multi-Sequen
main.py:    # run_plotcon() takes in 3 args: out_dir, out_clustalo and MSA output. Output: Plotcon graph and raw data filenames
```

Checking use of force

```
0
```

Checking use of plotcon

```
5
conservation_analysis.py:def run_plotcon(out_dir, out_clustalo, clustalo_output_filename):
conservation_analysis.py:# Function: Run plotcon command lines to generate Plotcon graph to terminal and save graph as .png fil
conservation_analysis.py:    plotcon_data = f"{out_clustalo}_plotcon_data"
conservation_analysis.py:    plotcon_graph = f"{out_clustalo}_plotcon_graph"
conservation_analysis.py:    print(f"Outputting {plotcon_graph} to terminal. Check graph, and close to continue!")
conservation_analysis.py:    return f"{plotcon_graph}.1.png", f"{plotcon_data}1.dat"
conservation_analysis.py:# Run plotcon command line to generate Plotcon raw data and save as .txt file.
conservation_analysis.py:# run_plotcon() takes in 3 args: out_dir, out_clustalo and MSA output.
conservation_analysis.py:    subprocess.call(f"plotcon -sequence {out_dir}/{out_clustalo}/{clustalo_output_filename} -graph dat
conservation_analysis.py:    subprocess.call(f"plotcon -sequence {out_dir}/{out_clustalo}/{clustalo_output_filename} -graph png
conservation_analysis.py:    subprocess.call(f"plotcon -sequence {out_dir}/{out_clustalo}/{clustalo_output_filename} -graph x11
main.py:from conservation_analysis import build_clustalo_input, run_clustalo, run_plotcon, get_consensus, get_infoalign
main.py:    plotcon_graph, plotcon_text = run_plotcon(out_dir, out_clustalo, clustalo_output)
main.py:    print(f"Plotcon output successfully generated! Generated {plotcon_graph}, {plotcon_text}")
main.py:    # run_plotcon() takes in 3 args: out_dir, out_clustalo and MSA output. Output: Plotcon graph and raw data filenames
main.py:    # With the Multi-Sequence Alignment(MSA) output, visualise data with Plotcon, through run_plotcon().
```

Checking use of blastp

```
# Function: Run blastp command line to conduct BLAST analysis of protein database with a protein sequence query. # Run BLAST an
```

Checking use of awk

Checking use of print()

```
50
  1 blast_analysis.py:    print("Generating scatterplot_blast.png...")
  1 blast_analysis.py:    print("Outputting Scatterplot of BLAST Hits to terminal. Check plot, and close to continue!")
  1 conservation_analysis.py:    print(f"Outputting {plotcon_graph} to terminal. Check graph, and close to continue!")
  1 fetch_sequence.py:# If query failure or no results, print error message and return an empty list.
  1 fetch_sequence.py:# If query failure, print error message and exit system.
  1 fetch_sequence.py:    print(f"Error fetching from NCBI Protein! Error: {response_efetch.status_code}")
  1 fetch_sequence.py:    print(f"Error querying NCBI Protein! Error: {response_esearch.status_code}")
  1 fetch_sequence.py:    print(f"No result for {query} on NCBI Protein!")
  1 main.py:    print(f"ClustalO output successfully generated! Generated {clustalo_output}")
  1 main.py:    print("Fetching all protein sequences in FASTA format...")
  1 main.py:    print(f"FASTA sequences successfully downloaded to {out_fasta}\n")
  1 main.py:    print(f"Generated prosite_scan_summary.tsv! PROSITE motifs found:\n{prosite_motifs}\n")
  1 main.py:    print(f"\nFull analysis successfully completed! Check {out_dir} for all outputs.\n")
  2 main.py:    print(f"\nInvalid input. {e}")
  1 main.py:    print(f"\nPlease input a valid protein family. {e}")
  1 main.py:    print(f"\nPlease input a valid taxon group. {e}")
  1 main.py:    print(f"\nSuccess! Your protein family: {pfam_name}\tPfam ID: {pfam_id}")
  1 main.py:    print(f"\nYour sample size: {conservation_analysis_size}")
  1 main.py:    print(f"\nYour scan size: {scan_size}")
  1 main.py:    print(f"Parsing {out_fasta}...")
  1 main.py:    print(f"Please input a number for the number of sequences to be analysed with ClustalO. Maximum: {len(records)
  1 main.py:    print(f"Please input a number for the number of sequences to be scanned for PROSITE motifs. Maximum: {conserva
  1 main.py:    print(f"Plotcon output successfully generated! Generated {plotcon_graph}, {plotcon_text}")
  1 main.py:    print(f"PROSITE output successfully generated! Generated {len(prosite_output)} .patmatmotifs files.")
  1 main.py:    print(f"Successfully completed BLAST analysis! Generated {blast_output}")
  1 main.py:    print(f"Successfully obtained consensus sequence! Generated {consensus}")
  1 main.py:    print(f"Successfully obtained infoalign results for {clustalo_output}! Generated {infoalign_results}\n")
  1 main.py:    print(f"Successfully parsed {out_fasta}\n")
  1 main.py:    print(f"Success! Your taxon group: {taxon_name}\tTaxon ID: {taxon_id}\n")
  1 main.py:    print(f"The {conservation_analysis_size} longest protein sequences will be used for conservation analysis.\n")
  1 main.py:    print(f"The {scan_size} protein sequences with the highest similarity to the consensus sequence will be scanne
  1 main.py:    print(f"Using {blast_ref} as BLAST reference sequence...")
  1 main.py:    print("\nInvalid input.")
  1 main.py:    print("\nInvalid input.")
  1 main.py:    print("Note: The larger the number, the slower the analysis. Recommended: 500-1000")
  1 main.py:    print("\nPlease input a valid email.")
  1 main.py:    print("\nRetrying...")
  1 main.py:    print("Performing conservation analysis with ClustalO and Plotcon...")
  1 main.py:    print("Protein UIDs successfully retrieved!\n")
  1 main.py:    print("Running BLAST on protein sequences...")
  1 main.py:    print("Scanning protein sequences for PROSITE motifs...")
  1 main.py:    print("Searching NCBI Protein...")
  1 main.py:    print("The top few sequences with the highest similarity with the consensus sequence will be scanned for PROSI
  1 main.py:    print("The top few sequences with the largest lengths will be used for conservation analysis.")
  2 main.py:    # Show user's input by printing these statements.
  1 main.py:    # When all inputs are valid, print these statements.
  1 scan_prosite.py:    print("Generating count_plot_motifs.png...")
  1 scan_prosite.py:    print("Outputting Count Plot of PROSITE Motifs Found to terminal. Check plot, and close to continue!")
```

Checking use of comments

```
126
  1 blast_analysis.py:    blast_df = pd.read_csv(f"{out_dir}/{blast_db}/{blastresults_filename}", sep = "\t", comment = "#", na
  1 blast_analysis.py:# Constructs a FASTA file containing all protein sequences.
  1 blast_analysis.py:# Function: Parse BLAST output file to a pandas dataframe, and visualise BLAST output by plotting a scat
  1 blast_analysis.py:# Function: Run blastp command line to conduct BLAST analysis of protein database with a protein sequenc
  1 blast_analysis.py:# Function: Run makeblastdb command line to construct a BLAST database.
  1 blast_analysis.py:# Function: Select the sequence with the highest % similarity with the consensus sequence and save the se
  1 blast_analysis.py:# make_blast_db() takes in 3 args: out_dir, list of dicts of all protein sequences and blast_db.
  1 blast_analysis.py:# Output: BLAST output file.
  1 blast_analysis.py:# Output: BLAST reference sequence FASTA file.
  1 blast_analysis.py:# Output: Custom BLAST database name.
  1 blast_analysis.py:# Output: Scatterplot of BLAST hits and parsed BLAST output to pandas dataframe.
  1 blast_analysis.py:# parse_blast_output() takes in 3 args: out_dir, blast_db and BLAST output file.
  1 blast_analysis.py:# run_blast_search takes in 3 args: out_dir, BLAST reference sequence FASTA file and blast_db.
  1 blast_analysis.py:# select_blast_ref() takes in 4 args: Top sequences in JSON, top sequences in infoalign results, out_dir
  1 blast_analysis.py:#!.venv/bin/python3
  1 conservation_analysis.py:# As per user's input size, sort the sequences by length in descending order, and select top few
  1 conservation_analysis.py:# build_clustalo_input() takes in 4 args: Parsed sequences to JSON, out_dir, out_clustalo and use
  1 conservation_analysis.py:# Function: Prepare an input FASTA file for ClustalO.
  1 conservation_analysis.py:# Function: Run clustalo command line.
  1 conservation_analysis.py:# Function: Run cons command line to obtain consensus sequence from MSA output
  1 conservation_analysis.py:# Function: Run infoalign command line to obtain % similarity of each sequence to consensus seque
  1 conservation_analysis.py:# Function: Run plotcon command lines to generate Plotcon graph to terminal and save graph as .png
  1 conservation_analysis.py:# get_consensus() takes in 3 args: out_dir, out_clustalo and MSA output.
  1 conservation_analysis.py:# get_infoalign() takes in 3 args: out_dir, out_clustalo and MSA output.
  1 conservation_analysis.py:# Output: Consensus sequence stored in FASTA format.
  1 conservation_analysis.py:# Output: infoalign results .txt file.
  1 conservation_analysis.py:# Output: Input FASTA filename for Clustalo.
  1 conservation_analysis.py:# Output: Multi-Sequence Alignment(MSA) output file with .clu.
  1 conservation_analysis.py:# Output: Plotcon graph and raw data files.
  1 conservation_analysis.py:# run_clustalo() takes in 3 args: out_dir, out_clustalo and input FASTA filename for ClustalO.
```

```
1 conservation_analysis.py:# Run plotcon command line to generate Plotcon raw data and save as .txt file.
1 conservation_analysis.py:# run_plotcon() takes in 3 args: out_dir, out_clustalo and MSA output.
1 conservation_analysis.py:#!.venv/bin/python3
1 fetch_sequence.py:# Else, return a list of unique IDs.
1 fetch_sequence.py:# Else, return the downloaded raw FASTA filename.
1 fetch_sequence.py:# Function: Query NCBI Protein database using Entrez esearch REST API to get a list of unique IDs.
1 fetch_sequence.py:# Function: With the list of unique IDs obtained from esearch, query NCBI Protein database using Entrez e
1 fetch_sequence.py:# If query failure or no results, print error message and return an empty list.
1 fetch_sequence.py:# If query failure, print error message and exit system.
1 fetch_sequence.py:# run_efetch() takes in 3 args: list of unique IDs, out_dir and email.
1 fetch_sequence.py:# run_esearch() takes in 3 args: protein family name, taxon ID and email.
1 fetch_sequence.py:#!.venv/bin/python3
1 input_handler.py:# Else, return the first Pfam ID and Pfam name.
1 input_handler.py:# Else, return the first taxon ID and taxon group name.
1 input_handler.py:# Function: Query EBI search with search term using REST API.
1 input_handler.py:# Function: Query NCBI Taxonomy database with search term using REST API.
2 input_handler.py:# If query failure or no results, raise an error.
1 input_handler.py:# validate_protein() takes in an arg: a protein family string name.
1 input_handler.py:# validate_taxon() takes in 2 args: a taxon group string name and email.
1 input_handler.py:#!.venv/bin/python3
1 main.py:    # build_clustalo_input() takes in 4 args: Parsed sequences to JSON, out_dir, out_clustalo and user's input for
1 main.py:    # build_prosite_input() takes in 3 args: Top sequences in JSON, out_dir, out_prosite. Output: List of input FA
1 main.py:    # Construct a BLAST database using makeblastdb, through make_blast_db().
1 main.py:    # Filter for the top sequences with high similarities to the consensus sequence, through parse_clu_results().
1 main.py:    # get_consensus() takes in 3 args: out_dir, out_clustalo and MSA output. Output: Consensus sequence FASTA file
1 main.py:    # get_infoalign() takes in 3 args: out_dir, out_clustalo and MSA output. Output: infoalign results .txt filena
1 main.py:    # If input > 2000 and exceeds recommended range, prompt user for a confirmation (yes/no). If invalid input, ra
1 main.py:    # If input is invalid, raise an error and reprompt user, until a valid protein family is inputted.
1 main.py:    # If input is invalid, raise an error and reprompt user, until a valid taxon group is inputted.
1 main.py:    # If list of unique IDs is empty, exit system with "no result" message.
1 main.py:    # If no, exit system. Else, continue.
1 main.py:    # If no, reprompt user for another integer. Else, continue.
1 main.py:    # If there are more than 1000 entries in the dataset, prompt user for confirmation.
1 main.py:    # Input needs to be an integer <= Number of selected sequences for conservation analysis. If invalid, raise an
1 main.py:    # Input needs to be an integer <= Total number of sequences. If invalid, raise an error and reprompt user, unt
1 main.py:# main() runs the pipeline.
1 main.py:    # Make an output directory for BLAST analysis and store the directory name as blast_db.
1 main.py:    # Make an output directory for ClustalO and store the directory name as out_clustalo.
1 main.py:    # Make an output directory for PROSITE scan and store the directory name as out_prosite.
1 main.py:    # Make an output directory named after protein family and taxon group. Store directory name as out_dir.
1 main.py:    # make_blast_db() takes in 3 args: out_dir, raw FASTA file of all protein sequences and blast_db. Output: Custo
1 main.py:    # Obtain consensus sequence with cons, through get_consensus().
1 main.py:    # Obtain % similarity of sequences to consensus sequence with infoalign, through get_infoalign().
1 main.py:    # Output: Summary table of PROSITE motif locations per sequence, count plot of PROSITE motifs found, and list o
1 main.py:    # Output: The top sequences in JSON and infoalign results.
1 main.py:    # parse_blast_output() takes in 3 args: out_dir, blast_db and BLAST output file. Output: Scatterplot of BLAST h
1 main.py:    # parse_clu_results() takes in 6 args: Parsed sequences to JSON, out_dir, out_clustalo, infoalign results file
1 main.py:    # parse_fasta() takes in 2 args: out_dir and raw FASTA filename. Output: All protein sequences in JSON format.
1 main.py:    # Parse output patmatmotifs files into a summary table, and plot a count plot of PROSITE motifs found, through
1 main.py:    # parse_prosite_output() takes in 3 args: out_dir, out_prosite and list of output patmatmotifs files.
1 main.py:    # Parse raw FASTA file into JSON format, through parse_fasta().
1 main.py:    # Perform conservation analysis with ClustalO, through build_clustalo_input() and run_clustalo().
1 main.py:    # Query NCBI Protein database using Entrez esearch to get a list of unique IDs, through run_esearch().
1 main.py:    r"^[a-zA-Z0-9.!#$%&'*+\/=?^_`{|}~-]+@[a-zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?(?:\.[a-zA-Z0-9](?:[a-zA-Z0-
1 main.py:    # Run BLAST analysis with blastp, through run_blast_search().
1 main.py:    # run_blast_search takes in 3 args: out_dir, BLAST reference sequence FASTA file and blast_db. Output: BLAST ou
1 main.py:    # run_clustalo() takes in 3 args: out_dir, out_clustalo and input FASTA filename for ClustalO. Output: Multi-Se
1 main.py:    # run_efetch() takes in 3 args: list of unique IDs, out_dir and email. Output: Raw FASTA filename.
1 main.py:    # run_esearch() takes in 3 args: protein family name, taxon ID and email. Output: List of unique IDs.
1 main.py:    # run_plotcon() takes in 3 args: out_dir, out_clustalo and MSA output. Output: Plotcon graph and raw data file
1 main.py:    # run_prosite_scan() takes in 3 args: out_dir, out_prosite and list of input FASTA files(filepaths) for patmat
1 main.py:    # Scan protein sequences for PROSITE motifs using patmatmotifs, through build_prosite_input() and run_prosite_
1 main.py:    # Select and construct a BLAST reference sequence FASTA file using the sequence with the highest similarity to
1 main.py:    # select_blast_ref() takes in 4 args: Top sequences in JSON, top sequences in infoalign results, out_dir and bl
2 main.py:    # Show user's input by printing these statements.
1 main.py:    # This while loop prompts the user for an email. If the email is invalid, reprompt the user, until a valid ema
1 main.py:    # This while loop prompts the user for a protein family, which is to be validated by validate_protein().
1 main.py:    # This while loop prompts user for an integer, for the number of sequences to be analysed with ClustalO.
1 main.py:    # This while loop prompts user for an integer, for the number of sequences to be scanned for PROSITE motifs.
1 main.py:    # This while loop prompts user for a taxonomic group, which is to be validated by validate_taxon().
1 main.py:    # This while loop prompts user for yes/no. If invalid input, raise an error and reprompt user, until valid.
1 main.py:#!.venv/bin/python3
1 main.py:    # Visualise BLAST output by plotting a scatterplot of BLAST hits (Alignment Length against % Identity), throug
1 main.py:    # When all inputs are valid, print these statements.
1 main.py:    # With the list of unique IDs obtained from esearch, query NCBI Protein database using Entrez efetch to get a
1 main.py:    # With the Multi-Sequence Alignment(MSA) output, visualise data with Plotcon, through run_plotcon().
1 parse_fasta.py:# Function: Parse the downloaded raw FASTA file with all sequences into JSON format.
1 parse_fasta.py:# Output: All protein sequences in JSON format.
1 parse_fasta.py:# parse_fasta() takes in 2 args: out_dir and raw FASTA filename.
1 parse_fasta.py:#!.venv/bin/python3
1 scan_prosite.py:# build_prosite_input() takes in 3 args: Top sequences in JSON, out_dir, out_prosite.
1 scan_prosite.py:    df = pd.read_csv(f"{out_dir}/{out_prosite}/{file}", sep = "\t", comment = "#", na_values = [""])
1 scan_prosite.py:# Function: As per user's input, filter and select the top few sequences with high % similarity to the con
1 scan_prosite.py:# Function: Parse output patmatmotifs files into a summary table, and plot a count plot of PROSITE motifs
1 scan_prosite.py:# Function: Prepare input FASTA files for patmatmotifs.
1 scan_prosite.py:# Function: Run patmatmotifs command line for each input FASTA file, to scan for PROSITE motifs.
1 scan_prosite.py:# Output: List of input FASTA files(filepaths) for patmatmotifs.
1 scan_prosite.py:# Output: List of output patmatmotifs files.
1 scan_prosite.py:# Output: Summary table of PROSITE motif locations per sequence, count plot of PROSITE motifs found, and l
1 scan_prosite.py:# Output: The top sequences in JSON and infoalign results.
1 scan_prosite.py:# parse_clu_results() takes in 6 args: Parsed sequences to JSON, out_dir, out_clustalo, infoalign results
```

```
1 scan_prosite.py:# parse_prosite_output() takes in 3 args: out_dir, out_prosite and list of output patmatmotifs files.
1 scan_prosite.py:# run_prosite_scan() takes in 3 args: out_dir, out_prosite and list of input FASTA files(filepaths) for pa
1 scan_prosite.py:#!.venv/bin/python3
```

Excellent.

- **Were any parts of the process multi-threaded?**
  NO!
  `blast` wasn't.
  `clustalo` wasn't explicitly (but fortunately defaults to all available threads anyway).
- **Did it use BioPython?**
  No, good! You found the workaround in `edirect` haha....!
- **Did it use Python modules (e.g. to interact with the OS)?**
  Yes, quite a few
- **Did it use Python pandas?**
  Yes, hence the env
- **Did it use Python plotting (as opposed to EMBOSS plotting)?**
  Yes

CODE

- **Was the user told what they can and can't do, preferably at or before the time they are doing it, by the code interface (rather than buried in the manual somewhere!?)**
  Not explicitly, but the relevant info was soon to hand.
- **esearch/efetch: is the search specific enough?**

```
egrep "esearch |efetch |xtract |efilter|query " *.py
```

```
blast_analysis.py:    subprocess.call(f"blastp -db {out_dir}/{blast_db}/{blast_db} -query {o
fetch_sequence.py:# Function: Query NCBI Protein database using Entrez esearch REST API to g
fetch_sequence.py:# If query failure or no results, print error message and return an empty
fetch_sequence.py:    query = f"({protein_family}) AND txid{taxon_id}[organism] NOT partial"
fetch_sequence.py:    params_esearch = {
fetch_sequence.py:    response_esearch = requests.get(f"{base_url}/esearch.fcgi", params = p
fetch_sequence.py:# Function: With the list of unique IDs obtained from esearch, query NCBI
fetch_sequence.py:# If query failure, print error message and exit system.
fetch_sequence.py:    params_efetch = {
fetch_sequence.py:    response_efetch = requests.post(f"{base_url}/efetch.fcgi", data = para
input_handler.py:# If query failure or no results, raise an error.
input_handler.py:# If query failure or no results, raise an error.
input_handler.py:    params_esearch = {
input_handler.py:    response_esearch = requests.get(f"{base_url}/esearch.fcgi", params = pa
main.py:    # Query NCBI Protein database using Entrez esearch to get a list of unique IDs,
main.py:    # With the list of unique IDs obtained from esearch, query NCBI Protein database
```

No, I dont think so! Need [Protein Name] in the query otherwise it looks in the whole record (or in this case, Pfam??) not just the protein description?

```
esearch -db protein -query "Aves[organism] AND glucose-6-phosphatase[Protein Name] NOT partial "
```

- **Correct: does the programme produce the correct output when run?**
  No (for reasons of the query being wrong), but the outputs ARE beautifully presented!
  I wasn't sure why `blast` was run?
- **Robust to error: does the programme check for valid inputs/outputs while processing, and if not good, fail gracefully with a useful error message?**
  Yes, I think it does.
- **Well-structured: is your code divided into logical functions/units, where that is possible?**
  Yes
- **Well-documented: do the comments help a programmer to understand/maintain your code?**
  Yes
- **Concise, well formatted: is your code laid out in a clear, consistent way?**
  Yes

Were the commands correct?

- *** initial processing stages**
  Ideally one wants to take advantage of all the `edirect` functionality, forming commands inside functions such as
  ```
  esearch_query="esearch -db protein -query \"{}[orgn] AND {}[Protein Name] NOT Partial\" | es
    xtract -pattern DocumentSummary -element AccessionVersion,Title,Organism,Slen".format(taxo
  ```

  and then, perhaps, read the fasta file into a dictionary, with the accession (or perhaps full header?) as the keys. The dictionary could then also be used for piping sequences as STDIN into motif searching, etc....
  Alternatively, this information could be brought into a pandas dataframe and then used throughout for the rest of the processing.

  Yes, however, query is not quite right in terms of specificity
  Issues with the order things done (`clustalo` / `cons` / `blast`)
- *** final processing stages**
  Couldnt change any programme parameters for `plotcon`
  `patmatmotifs` done, but wildcard not done (or was that the `blast` bit?!)
  `patmatmotifs` sequences were limited, why?
  The files in the motif directory have recogniseable names, and good to see a summary for the whole set!
  No wild card app (or the blast?)
- **Was there any two-way interaction with the user?**
  Yes, some
- **If so, were there error traps for input?**
  Yes, lots
- **If so, was all the input requested at the beginning, or did the user have to enter info at different stages as the programme was running?**
  Staged, which isn't an issue with the small dataset, but for larger ones, it meant I had to be there to wait for it.
  It might make sense to have the parameters supplied for everything at the beginning, though most that I would like to have changed were not given to me!
- **Was the user ever given the option to continue or not continue?**
  Yes
- **Did the user have to do anything other than enter the taxonomic group and protein family?**
  Yes, make choices
- **Did the programme check how many species were in the requested dataset?**
  Maybe, but did we ever get told how many sequences?!
- **Did the programme tell us what the species were?**
  No
- **Did the programme check to see the size range of the requested dataset?**
  No, and because the search wasn't specific enough, some were very clearly not "the right thing". This actually worked against you, most of the "big ones" were not the real thing, so when you said you were keeping the top 500, they were probably the worst ones to keep...
- **Did the programme let us filter on size?**
  Not really, just ranked
- **Were there any progress indicators?**
  Yes
- **This code was first tried with Aves glucose-6-phosphatase. How did it cope with the bigger dataset (Eukaryota kinase)?**
  Took quite a while!
- **Was an alignment done irrespective of the number of sequences in the initial set?**
  Yes, but the user was told there were a lot of sequences, so they did have the option to bail out.
  A consensus route had been taken too...

- **Was the number of sequences reduced?**
  "Keep the biggest"
  `blast` with `cons` as query was a sort of surrogate too?
- **Was the `cons` programme used, and what parameters were chosen to generate it (e.g -plurality, -identity, -setcase)?**

```
grep -w cons *.py
```

```
conservation_analysis.py:# Function: Run cons command line to obtain consensus sequence from
conservation_analysis.py:    subprocess.call(f"cons -sequence {out_dir}/{out_clustalo}/{clus
main.py:    # Obtain consensus sequence with cons, through get_consensus().
```

Yes, but NO parameters given....so the consensus is...err... not good; the use of `cons` should come with a health warning for the user, or the ability to modify the parameters, as they can make a HUGE difference.

SMALL DATASET CONSENSUS

```
cat ./PAP2_Aves/PAP2_Aves_clustalo/PAP2_Aves_clustalo_consensus.fasta
```

```
EMBOSS_001
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

```
Molecular weight = 295617.48            Residues = 2508
Residue          Number          Mole%           DayhoffStat
X = Xaa          2059            82.097          82097.285
```

If you really had to use `blast`, how about all against all, then take the top 250 highest non-self hits (use -outfmt 6 or -outfmt 7), THEN do the alignment etc.
Or `clustalo` to make a distance matrix, chose the appropriate one as query for `blast`, then take the best 250 HSP scores.

- **Were all the outputs kept?**
  Yes, and all put in sensible folders. =-) THANK YOU!

What were the analyses offered?

- **Conservation:**
  `plotcon` was used to produce the plot, with default parameters only (window size can make a big difference....)
  No explanation of `plotcon` output? What ARE the values on the axes?
  Graph not labelled for these data; underlying data were not assessed at all?
- **Motif search:**
  `patmatmotifs` was identified as the correct EMBOSS programme to use, with default parameters only, but not permitted for all sequences!!
  `patmatmotifs` needs to be done for all sequences (not all will have the motifs, depends on their similarities)
  there are many different outputs possible, for example `-rformat excel` makes a nice report...
  Report files provided with the accession IDs were in the file names, so I would know which ones I might want to open.
  Outputs were made into a "catalog" of what was found: good! Frequency plotted too!
- **wildcard option(s):**
  `blast?` were done, but from the on-screen interaction, I wasnt clear what it was doing for me! How did these outputs relate back to the conservation plot?

Did the programme meet all the requirements of the assignment

- **to identify a family of protein sequences from a user-defined subset of the taxonomic tree that could then be processed using, for example, one or more of the EMBOSS programmes installed on the MSc server**
  yes, but minor concerns about specificity
- **to determine, and plot, the level of sequence conservation**
  to show the plot, could have used `display` or `eog` or `xdg-open` or `firefox` or `image` from matplotlib
  for the file that was saved, or just use the `X11` option from `plotcon`...
  plotted and saved, but output data not processed much further/summarised, couldnt change any parameters
- **to scan protein sequence(s) with motifs from the PROSITE database, to determine whether any known motifs (domains) are associated with this subset of sequences**
  Yes, but not for all....
- **to do any other appropriate EMBOSS (or other) analysis that you think might add relevant biological information to the outputs**
  Not sure what to put here!

The user "experience"

- **Utility: does your programme present a seemingly useful tool for biologists?**
  Yes, with some tweaks as noted above re when to use (or not) `blast` and `clustalo`, stop using `consensus`!!!
- **Usability: how easy is your programme to use?**
  Easy once I RTFM...
- **Interface: does your programme present a consistent interface to the user?**
  Yes, nice interaction

The user manual, which contains instructions for running the program, and interpreting the output

Write a "help manual" for your programme, which has two main sections;

- a section aimed at an "ordinary user": it contains instructions for running the program, and interpreting the output; it is aimed more at biologists who may not know much about Python3 coding, they just want the outputs!

  The user section of manual **doesn't** need to be huge. Remember that this is for a non-programmer to read, so you don't have to go into details of what variables you're using, *etc.*. It just has to describe briefly how the programme works, how to use it, and, perhaps, how NOT to use it!?

  You shouldn't need more than 5 pages of text, and probably fewer would be fine. Pictures are good here. <u>You should use the outcomes from the test set as example inputs and outputs in this manual</u>.
- a section aimed at a competent Python3 code-writer. This is usually called a "maintenance manual", which explains/shows how the different parts of the programme fit together. Feel free to use words like variable, function, iteration, error trap etc!

  This bit of the help manual should essentially be a description of the programme components and how they link together, **not** a listing of variables/functions/dictionaries/lists etc. Flow diagrams work very well here.

  You shouldn't need more than 5 pages of text, and probably fewer would be fine. Pictures are good here too.

Good to see a summary overview at the beginning so the user can judge whether they want to use the software in the first place!

- **Readability of documentation: is the biologist user manual section easy to read and understand?**
  Yes, the user is led through!
- **General description: is it easy for the user to understand how to run the programme?**
  Yes
- **indicates any things the user will have to do to make things work**
  Yes, clear instructions (but the user has to read them!!)
- **Is there test set example output shown?**
  Yes, from all stages.

The maintenance manual section, which explains how the different parts of the programme fit together

It is aimed at a programming-literate audience (so you should feel free to use words like variable, function, iteration, error trap etc); this should essentially be mostly a description of the workflow components and how they link together, rather than a listing of variables/functions/dictionaries/lists etc.

- **Readability of documentation: is the maintenance manual easy to read and understand?**
  Yes, excellent.
- **General description: is it easy for a programmer to understand how the programme works?**
  This was also excellent.
- **highlights any difficulties, if any, that you have come across**
  Not asked for, and none mentioned.

Any final comments

Excellent effort! Well done!
I hope you find at least some of this feedback helpful and constructive? I also hope you enjoyed doing this as much as I did

looking at it!
If there are things that you want to chat about, as you know, please do just get in contact.
Cheers,
al